

Napomena: skripta je sklona ispravkama i možda ima grešaka i nejasnih pojmova :) srećno učenje <3

LEKCIJA 1: Uvod i arhitektura ML kompajlera

1. Zašto su nam potrebni specijalizovani ML prevodioci?

Tradicionalni kompajleri (poput standardnog LLVM-a) dizajnirani su za rad sa C-olikim jezicima i CPU arhitekturama opšte namene. Oni imaju dva ključna ograničenja koja ih čine neupotrebljivim za duboko učenje:

- Ograničenje reprezentacije: Tradicionalni kompajleri barataju niskonivojskim instrukcijama. Oni ne mogu da prepoznaju strukture visokog nivoa (npr. ne znaju šta je matrično množenje ili konvolucija), pa samim tim ne mogu ni da ih optimizuju.
- Monolitski dizajn: Veoma su kruti; dodavanje podrške za potpuno nove hardverske akceleratori (GPU, TPU, NPU) zahteva prepisivanje ogromnog dela kompajlera.

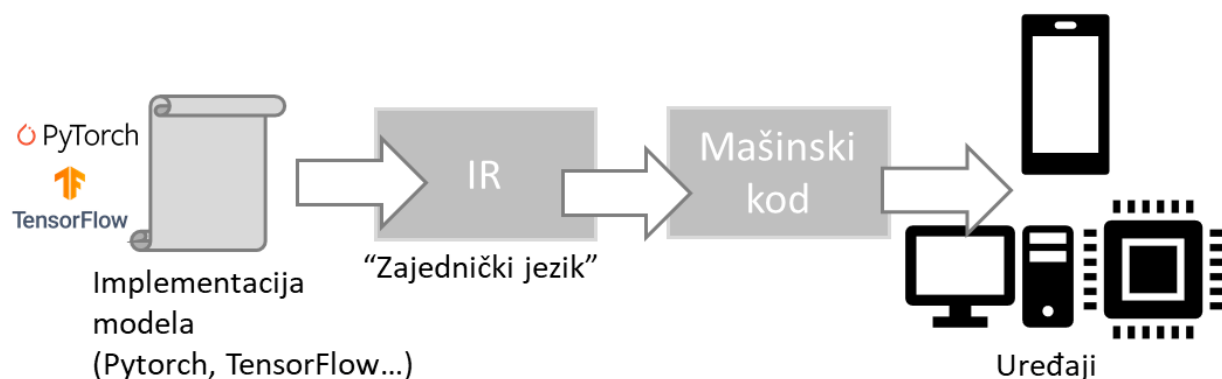
2. Problem "M prema N" ($M \times N$)

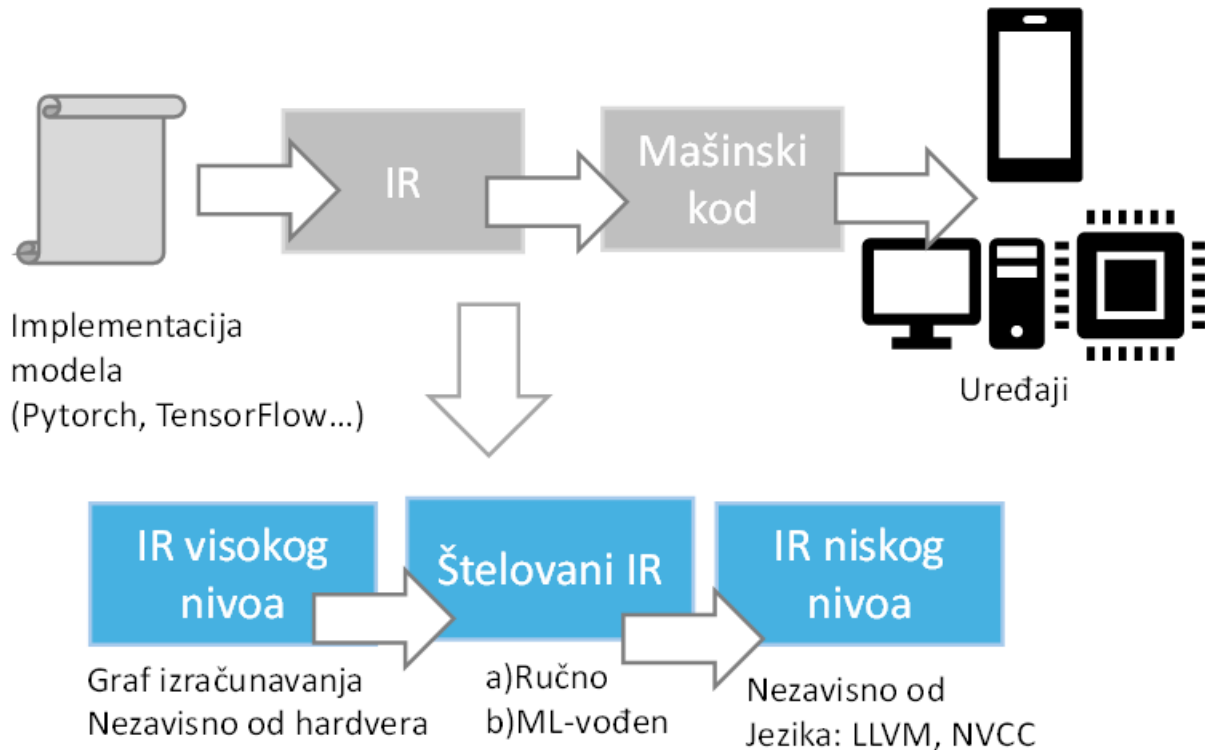
U svetu mašinskog učenja postoji:

- M različitih razvojnih okvira (Framework-a): PyTorch, TensorFlow, JAX, MXNet...
- N različitih hardverskih platformi: Nvidia GPU, Intel CPU, ARM (mobilni), Google TPU, AMD...

Ako bismo išli tradicionalnim putem, morali bismo da napišemo poseban prevodilac/optimizaciju za svaku moguću kombinaciju ($M \times N$ softverskih paketa).

Rešenje: ML kompajleri uvode Međukod (IR - Intermediate Representation). Svi framework-ci se prevode u jedinstveni međukod, a onda se taj međukod prevodi u mašinski kod za bilo koji hardver. Time se $M \times N$ problem svodi na jednostavniji $M + N$ problem.





Polazeći od koda modela mašinskog učenja, prevodioci generišu međukod visokog i niskog nivoa pre nego što se generiše kod direktno izvršiv na ciljanoj hardverskoj platformi.

Za generisanje koda polazeći od IR, prevodioci se oslanjaju na generatore koda, pri čemu je jedan od popularnijih LLVM, koji koristi MLIR (okvir za prevodjenje) za razvoj drugih prevodilaca. Ovaj proces se naziva snižavanjem, zato što se kod visokog nivoa kompajlera svodi na direktno izvršiv kod niskog nivoa za ciljani hardver.

IR visokog nivoa za modele mašinskog učenja najčešće predstavljaju takozvane grafove izračunavanja. Oni služe za razumevanje svojstava samog modela, na osnovu kojih je dalje moguće izvršiti optimizaciju. Nad IR reprezentacijama visokog nivoa se primenjuju tehnike globalne optimizacije. Ove optimizacije se mogu realizovati kompletno ručnim štelovanjem, a sve češća je i primena tehnika mašinskog učenja i predikcija sa ciljem efikasnog pronalaženja optimalnog grafa za ciljani hardver.

3. Glavni ciljevi prevođenja i zahtevi u produkciji

- **Inference (Zaključivanje):** Pokretanje već obučenog modela na ciljanom hardveru radi predikcije. Zahteva minimalno kašnjenje (latency), visoku propusnost (throughput) i malu potrošnju energije.
- **Prilagođavanje hardveru:** Generisanje koda koji maksimalno koristi memorijsku hijerarhiju (keš, registre) i paralelizam specifičnog čipa.

- Razvoj biblioteka matematičkih operatora: Automatizacija pisanja operatora umesto ručnog pisanja.

4. Opšti proces prevođenja modela

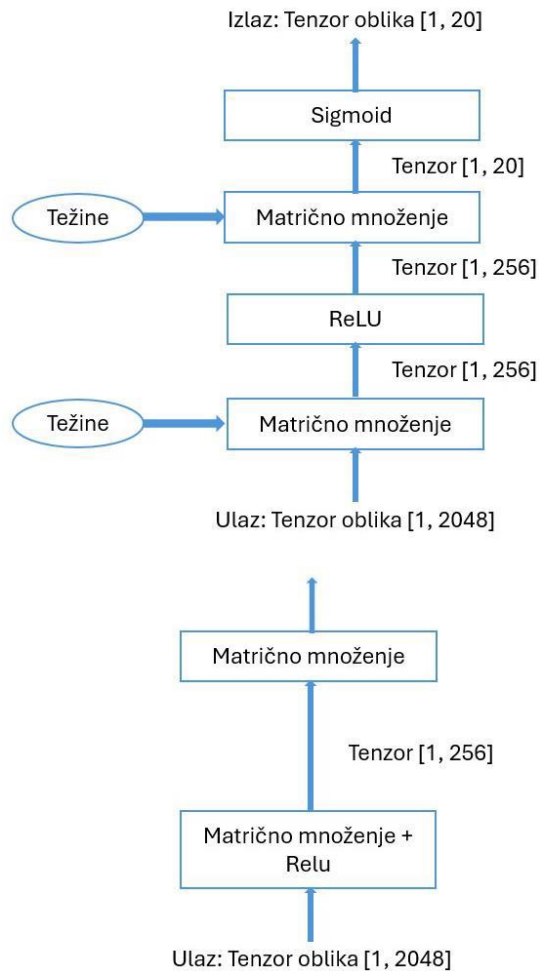
Proces ide kroz strogu hijerarhiju nivoa apstrakcije:

1. Frontend: Učitavanje modela iz PyTorch/TensorFlow i konverzija u graf visokog nivoa.
2. High-Level IR (Graf izračunavanja): Optimizacije na nivou celog modela (fuzija, uklanjanje mrtvog koda) – visokonivojske optimizacije.
3. Low-Level IR (Petlje): Pretvaranje operacija u konkretne petlje (TIR/MLIR) i optimizacija rasporeda memorije – niskonivojske optimizacije.
4. Backend / CodeGen: Generisanje sirovog mašinskog koda (LLVM bitcode, CUDA kod, ASM) za specifičan hardver.

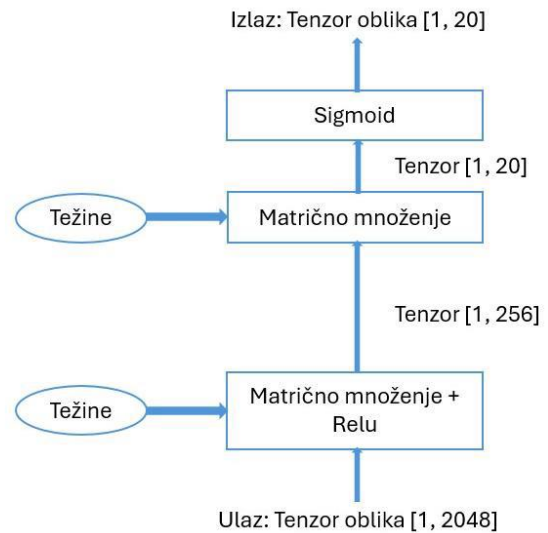
5. Osnovni pojmovi

- Graf izračunavanja (Computational Graph): Usmereni aciklični graf (DAG) gde čvorovi predstavljaju operacije (operatore), a grane predstavljaju tok podataka (tenzore).
- Tensor: Višedimenzionalni niz podataka sa definisanim oblikom (shape) i tipom podataka (dtype, npr. float32) – osnovni element u izvršenju modela mašinskog učenja; u njega se smeštaju međurezultati kao i ulaz i izlaz izvršavanja modela.
- Tenzorske funkcije: Matematičke definicije operacija nad tenzorima (operacije nad elementima poput ReLU, redukcione poput Sum, ili kompleksne poput Conv i MatMul); kao ulaz primaju tenzor i kao izlaz vraćaju tenzor, koji je dobijen primenom definisane matematičke operacije na ulazni tenzor; tenzorska fja može biti jedno matematičko izračunavanje a može biti i čitava sekvenca matematičkih operacija.

Oblik modela u fazi razvijanja:



Oblik modela u fazi produkcije:



```

def linear_relu(x, w, y):
    for i in range(1):
        for j in range(256):
            y[i, j] = 0
            for k in range(2048):
                y[i, j] = x[i, k] * w[k, j]
            y[i, j] = max(out[i, j], 0)
  
```

Može se primetiti da funkcija “matrično množenje + ReLU” može da se predstavi na više načina (kao pravougaonik u grafu ili kao kolekcija ugnjeđenih petlji). Ti načini za reprezentaciju tenzorske funkcije su apstrakcije tenzorskih funkcija. Prevođenje u mašinskom učenju se može posmatrati i kao vid transformisanja tenzorskih funkcija predstavljenih pomoću jedne ili više apstrakcija. Svaka apstrakcija tenzorskih funkcija poseduje svoj skup optimizacija koje se mogu primeniti na njih.

6. Optimizacije modela mašinskog učenja

Nakon “snižavanja” koda sa ciljem izvršenja na odabranoj hardverskih platformi, može doći do problema sa performansama. Opšta rešenja za ovu svrhu imaju mogućnost da IR snize na kod ciljanog hardvera, ali su performanse tokom izvršenja često ispod očekivanja. Ovakva situacija se objašnjava činjenicom da generisani kod za željeni hardver ne iskorišćava dovoljno njegove prednosti i specifičnosti, s obzirom da aspekti poput načina pristupa podacima i hardverski keš nisu uzeti u obzir, niti napredne mogućnosti poput hardverski realizovanih vektorskih i paralelnih

operacija izračunavanja. Cilj prevodilaca mašinskog učenja je, između ostalog i da premoste jaz između implementacije modela i samog hardvera.

Prevodioci za mašinsko učenje obično imaju dve zasebne komponente:

- optimizacija - prilagođavanje modela za dati hardver da bi se postigli benefiti po pitanju performansi, i
- snižavanje - generisanje koda za dati hardver.

Postoje lokalne i globalne optimizacije. Lokalne obuhvataju optimizaciju skupa operatora koji se javljaju unutar razmatranog modela (vektorizacija, paralelizacija, deljenje petlji (loop tiling), fuzija operatora...), dok se globalne optimizacije odnose na graf izračunavanja u celini (horizontalna (spaja slojeve koji su jedan pored drugog) i vertikalna (spaja slojeve koji su jedan ispod drugog) fuzija).

7. Brzi pregled industrijskih rešenja (Za prepoznavanje)

- XLA (Accelerated Linear Algebra)
- PyTorch Glow
- IREE
- NVCC (NVIDIA CUDA Compiler)
- Torch-MLIR
- Apache TVM
- Triton

LEKCIJA 2: Uvod u Apache TVM i osnovni koncepti

1. Šta je Apache TVM?

TVM je open-source softverski stek za prevođenje i optimizaciju modela mašinskog učenja sa bilo kog framework-a na bilo koju hardversku arhitekturu (CPU, GPU, FPGA, ASIC).

2. Glavni ciljevi i prednosti TVM-a

TVM učitava model iz postojećih okvira dubokog učenja (PyTorch, TensorFlow) i generiše optimizovan kod niskog nivoa za raznovrstan skup hardverskih akceleratora.

Prednosti:

1. Visoke performanse: Optimizuje modele na dva nivoa - na nivou grafa (visoki nivo) i na nivou samih tenzorskih operacija (niski nivo).

2. Široka kompatibilnost: Podržava sve vodeće ML okvire i hardware.
3. Proširivost: Omogućava inženjerima da ručno dodaju svoje optimizacione prolaze (passes) i registruju nove hardverske akceleratora.
4. Jednostavno korišćenje: Automatizovano upravljanje optimizacijom preko ugrađenih alata.

3. Arhitektura i ključne komponente TVM-a

TVM treba da reši 2 glavna izazova:

1. Maksimalno iskoristiti specifične mogućnosti hardverskih akceleratora;
2. Generisati efikasan kod bez ručnog podešavanja operatora.

TVM prevazilazi ove izazove uvođenjem 3 ključna modula:

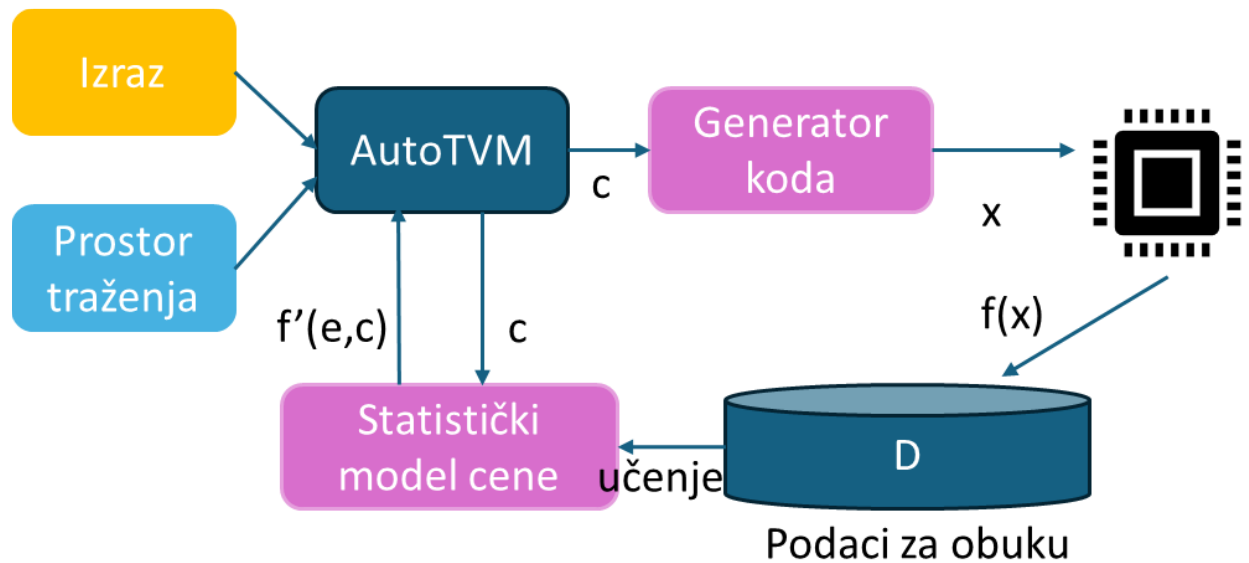
1. Jezik specifičnog domena za tenzorska izračunavanja: Jezik kojim se obezbeđuju primitive za transformaciju tenzorskih programa koje generišu različite verzije tog programa sa različitim optimizacijama. U ove jezike spada TensorIR i Tensor Expression DSL
2. Automatizovani okvir za optimizaciju programa (AutoTVM): Koristi model mašinskog učenja koji pretražuje prostor mogućih varijanti optimizacija tenzorskog operatora i bira onu koja je pogodna za odabrani hardverski akcelerator.
3. Transformator grafova izračunavanja: Transformator grafova primenjuje optimizacije visokog nivoa i optimizacije na nivou operatora na dati graf izračunavanja i proizvodi novi, optimalniji graf.

Opšti tok prevođenja: model se učitava iz okvira u međukod visokog nivoa -> modul 3) spoji operacije globalno -> modul 1) DSL ponudi gomilu načina kako te spojene operacije mogu da se zapišu kroz petlje tj. kroz primitive -> modul 2) kroz mašinsko učenje isproba sve petlje na hardveru i uzme najbolju po brzini -> prevodilac željenog hardverskog akceleratora generiše mašinski kod.

AutoTVM pronalazi optimalni graf izračunavanja modela u sledeća 4 koraka:

1. Nalaženje čvorova u grafu koji se mogu štelovati (sa tenzorskim operatorima)
2. Predikcija vremena izvršenja korišćenjem mašinskog učenja za svaki čvor

3. Pronalaženje najbrže putanje izvršenja za svaki od čvorova
4. Spajanje najbržih putanja i čvorova da bi se izvršio ceo graf za najkraće vreme



TVM koristi više vrsta apstrakcija za reprezentaciju tenzorskih funkcija. Na višem nivou tu je Relay IR koji se koristi za reprezentaciju grafova izračunavanja na uniforman način. Na nižem nivou su Tensor Expression DSL i TensorIR. Tensor Expression DSL služi da opiše tenzorska izračunavanja u čistom funkcionalnom jeziku (bez bočnih efekata), dok TensorIR predstavlja ta izračunavanja na nivou ugnježđenih petlji, višedimenzionalnih bafera i pojedinačnih izraza.

IRModule – prvi ključni pojam TVM steka

- **Definicija:** Centralna struktura podataka u TVM steku. Predstavlja kolekciju tenzorskih funkcija.
- **Uloga:** Kada se model učitava u TVM, on se **odmah prevodi u IRModule** i ceo model je sadržan u njemu (graf izračunavanja, tenzorski programi i pozivi spoljnih biblioteka).
- **Tipovi funkcija unutar IRModule** (zavise od nivoa apstrakcije u kome se model trenutno nalazi):

1. `relax::Function` /* noviji međukod */ (ili `relay::Function` /* stariji međukod */): Funkcionalni graf izračunavanja visokog nivoa (sadrži kontrolu toka, rekurziju, strukture).
2. `tir::PrimFunc`: Niskonivojski tenzorski program operatora (sadrži petlje, niti, vektorske instrukcije, memorijske bafere).

Načini kreiranja IRModule:

A. Učitavanje iz postojećih modela (PyTorch / ONNX)

B. Kreiranje preko Relax NN modula

C. Direktno pisanje preko TVMScript-a

Opšti tok prevođenja (High-Level Pipeline)

Proces se odvija kroz 4 stroge faze:

1. **Učitavanje (Frontend)**: Model se uvodi iz eksternog framework-a i pretvara u početni IRModule.
2. **Transformacija (Optimizacija)**: IRModule se progresivno transformiše u drugi, funkcionalno ekvivalentni ali brži IRModule.
3. **Kompilacija za hardver (CodeGen)**: Finalni IRModule se prevodi u runtime.Module specifičan za izabrani prevodilac.
4. **Izvršavanje (Runtime execution)**: Korisnik učitava runtime.Module i pokreće funkcije u aplikaciji (podržani jezici: Python, C++, Rust, Go...).

Transformacije – drugi ključni pojam TVM steka

Služe za optimizaciju i spuštanje IR modela sa višeg na niže nivoe apstrakcije. Dele se na:

- **Relax/Relay transformacije**: Optimizuju **graf izračunavanja** (visoki nivo). Primeri: izračunavanje konstanti u vreme kompilacije, eliminacija mrtvog koda, fuzija operatora, izmena rasporeda tenzora.

- **TIR transformacije:** Optimizuju i prevode **TensorIR programe** (niski nivo). Primeri: svođenje višedimenzionalnih adresa na jednodimenzionalne, pojednostavljenje indeksa petlji

MetaScheduler (Optimizacija mašinskim učenjem)

Pronaći idealne petlje ručno ili preko heuristike je nemoguće. Rešenje je **MetaScheduler** (koji je zamenio stari *AutoTVM*, kojem je donekle trebala pomoć inženjera), koji je potpuno optimizovan:

1. **Prostor pretraživanja:** Definiše se skup akcija za izmenu petlji, tzv. **primitive raspoređivanja** (*scheduling primitives* - npr. vektorizacija, splitovanje, umetanje). One čine prostor pretrage.
2. **Pretraga vođena ML-om:** Sistem pomoću algoritama mašinskog učenja pronalazi najbolju kombinaciju primitiva.
3. **Snižavanje:** Na osnovu poredničke konfiguracije, model se automatski spušta u maksimalno brz niskonivojski TensorIR.

LEKCIJA 3: MLIR i Torch-MLIR

1. Šta je MLIR?

MLIR (*Multi-Level Intermediate Representation*) je kompajlerska infrastruktura (podprojekat LLVM-a) koja rešava problem fragmentacije međukodova. Omogućava postojanje i preplitanje više nivoa apstrakcije u istom modulu.

2. Dijalekti (Dialects) i operacije u MLIR-u

Dijalekt je samostalni, modularni podjezik unutar MLIR-a koji definiše sopstvene operacije, attribute i tipove podataka, prilagođene specifičnom nivou apstrakcije. Dijalekti mogu slobodno komunicirati i postepeno se prevoditi iz jednog u drugi procesom snižavanja (*lowering*).

Kako kompajler prolazi kroz dijalekte:

1. Visoki nivo (top-level): predstavljaju ceo graf -> `torch.linear()`
2. Srednji nivo (Linalg/TOSA) – za linearnu algebru
3. Niski nivo (SCF/Affine) – sve se pretvara u petlje -> `scf.for`
4. Najniži nivo (LLVM dijalekti) – kod skoro identičan mašinskom

Primeri dijalekata:

- Tensorflow: predstavlja operacije višeg nivoa u Tensorflow biblioteci, npr. Conv2D, MatMul i ReLU
- Affine: daje operacije za opis "afinih" transformacija, što ovde znači bilo kakvo preslikavanje nekoliko tenzora u ugnježdene petlje
- Math: obezbeđuje osnovne matematičke operacije
- LLVM: ima skup instrukcija koji se može direktno dalje prevesti u LLVM međujezik

Operacije su osnovne jedinice sastava koda u MLIR-u i svaka ima ime, spisak atributa, ulazne i izlazne tipove i rezultate.

Dijalekti imaju međuoperabilnost, i mogu se prevesti jedan iz drugog tako što dodajemo prolaze (pass-ove). Upravo tako postizemo prevod iz koda višeg nivoa u specijalizovaniji kod po potrebi.

Svaki prolaz vrši određenu transformaciju ili analizu nad MLIR programom. Svaki prolaz deluje nad jednim modulom, koji sadrži skup funkcija. Ove funkcije su podeljene u regione koji predstavljaju blokove operacija. Tipovi prolaza su: optimizacijski prolazi (spajanje petlji, eliminacija mrtvog koda, deljenje petlji), spuštajući prolazi (prevođenje iz jednog dijalekta u drugi), i prolazi za generisanje koda.

Dosta strukture u MLIR-u je pruženo uz "regione". Region je skup operacija koje predstavljaju blok izvršavanja. Regioni nam pružaju struktuiran način da organizujemo operacije i lakše sprovodimo optimizacije.

Napomena: pogledati primere iz materijala

3. Torch-MLIR

Torch-MLIR služi kao srednji sloj između PyTorch-a i ciljanog hardvera. Omogućava sistemsku konverziju modela iz PyTorch okruženja u MLIR format.

Sastoji se iz dva ključna dela:

1. Frontend – interfejs ka PyTorch-u
2. Backend – dalja obrada i spuštanje IR-a iz Torch-MLIR dijalekta u ciljane dijalekte

Ova dva sloja međusobno komuniciraju pomoću backend contract sloja

Kada Torch-MLIR uvozi model iz PyTorch-a, on se oslanja na 3 interfejsa:

1. TorchScript: Pretvara PyTorch kod u statički graf izračunavanja nezavisan od Python runtime-a.
2. TorchFX: Alat za dinamičku transformaciju grafa na nivou Python-a.

3. LazyTensor: Mehanizam koji odlaže izvršavanje operacija dok se ne sakupi veći blok (graf).

4. Optimizacioni prolazi (passes) kroz graf u Torch-MLIR-u

Nakon uvoza u inicijalni Torch dijalekt, graf prolazi kroz sledeće faze čišćenja:

- InlinerPass: Zamenjuje pozive eksternih funkcija njihovim stvarnim telom koda.
- CanonicalizerPass: Svodi matematički ekvivalentne operacije na jedinstvenu, standardnu formu.
- CSEPass (Common Subexpression Elimination): Pronalazi identična izračunavanja u grafu i zamenjuje ih jednom promenljivom tj. izračunatom vrednošću (brisanje duplog posla).
- Shape Inference Pass (Određivanje dimenzija): Algoritam koji prolazi kroz graf i izračunava statičke dimenzije i tipove za svaki međutenzor.
- Decompose Complex Op (Dekompozicija): Razbijanje složenih operacija na jednostavnije.
Primer: Linear sloj se dekomponuje u Transpose -> MatMul -> Add (za bias).

5. Tri krajnja MLIR dijalekta (Backend Contract)

Nakon optimizacija, kod se prevodi u jedan od 3 podržana standardna dijalekta:

1. Linalg dijalekat: Čista linearna algebra. Eksplicitno razdvaja ulazne i izlazne bafere. Odličan za CPU/GPU.
2. TOSA dijalekat (ARM): Standardizovan skup tenzorskih operatora za mobilne, ugrađene i krajnje uređaje.
3. StableHLO dijalekat (Google): Visokonivojski dijalekt zasnovan na XLA kompajleru. Koristi se za cloud infrastrukturu i Google TPU akceleratoru.

6. End-to-End (E2E) Testing Framework

Pošto je krajnji proizvod Torch-MLIR-a novi međukod, njegova ispravnost se proverava preko E2E testiranja. Isti ulazni tenzor se prosleđuje originalnom PyTorch modelu i novom MLIR kodu koji se izvršava u referentnom runtime-u (npr. IREE). Ako su izlazne vrednosti unutar numeričke tolerancije, IR je ispravan.

Generalni tok prevodjenja: Torch dijalekat -> čišćenje/globalne optimizacije -> određivanje dimenzija (shape inference) -> dekompozicija -> spuštanje na jedan od tri dijalekta -> E2E testiranje

LEKCIJA 4: Apache TVM - Relax IR

1. Zašto Relax umesto starog Relay IR-a?

- Relay je bio čisto funkcionalni jezik i modelovao je graf isključivo kao stablo ugnježđenih izraza. To je činilo optimizacionu logiku krhkom i komplikovanom. On je super radio za starije mreže čije su dimenzije slika i tenzora bile fiksne. Međutim dinamičke dimenzije je tretirao preko nepouzdatih "unknown" anotacija.
- Relax (*Relaxing Graph Optimizations*) je funkcionalno-imperativni jezik – dopušta uređeno ponašanje sa bočnim efektima. Donosi dve ključne inovacije:
 1. Prvoklasna podrška za dinamičke oblike: Tretira dimenzije kao simboličke objekte koji se mogu menjati i pratiti tokom izvršavanja (npr. oblik $[1, n]$).
 2. Kros-nivojska (cross-level) integracija: Dozvoljava da unutar istog modula (IRModule) istovremeno žive i graf visokog nivoa i sirovi niskonivojski TensorIR kod sa petljama.

Relax prisilno prevodi kod u Administrativnu normalnu formu (ANF). Pravilo ANF-a glasi: *Svaka operacija mora biti izvršena u zasebnom redu i njen rezultat mora odmah dobiti jedinstveno ime u obliku lokalne promenljive.*

- *Svrha*: Izbacuje ugnježdene izraze. Omogućava kompajleru da tačno zna životni vek svakog bafera i primeni optimalno upravljanje memorijom.

Dataflow blokovi su specijalno izolovane sekcije unutar Relax funkcije koje sadrže isključivo operacije bez bočnih efekata (čist tok podataka unutra-ka-spolja, bez alokacija memorije, petlji ili grananja). Kompajler bezbedno vrši fuziju operatora isključivo unutar ovih blokova.

Fuzija operatora (Operator Fusion) spaja više odvojenih operacija u jednu zajedničku hardversku petlju, čime eliminiše upisivanje međurezultata u RAM.

Ključne karakteristike Relax IR:

1. Prvoklasni simbolički oblik: Relax koristi simboličke oblike za predstavljanje dimenzija tenzora. Ovi oblici omogućavaju globalno praćenje dinamičkih odnosa dimenzija između tenzorskih operacija i poziva funkcija.
2. Višenivovska apstrakcija: Relax podržava apstrakciju na više nivoa, od visokih slojeva neuronskih mreža do niskih tenzorskih operacija. Ovo omogućava optimizacije koje prelaze različite hijerarhije unutar modela. To znači da se optimizacije ne primenjuju samo na osnovnim operacijama tenzora (poput sabiranja ili množenja), već i na višim nivoima, kao što su slojevi neuronskih mreža.

3. Kompozabilne transformacije: Relax nudi okvir za kompozabilne transformacije koje se mogu selektivno primeniti na različite komponente modela. To uključuje mogućnosti poput parcijalnog snižavanja i parcijalne specijalizacije.

2. Ključni elementi

- Informacije o strukturi: `tvm.relax.StructInfo` – osnovna apstraktna klasa, klase koje opisuju specifične structure - `ObjectStructInfo`, `PrimStructInfo`, `ShapeStructInfo`, `TensorStructInfo`, `TupleStructInfo`
- `R.call_tir`: nova apstrakcija u Relax-u koja omogućava pozivanje primitivnih tenzorskih funkcija (`TensorIR` funkcija) unutar istog `IRModule`-a. To znači da se u istom modulu mogu kombinovati Relax i Tensor IR funkcije. Ovo je ključna karakteristika Relax-a koja omogućava međunivovske apstrakcije, od visokonivovskih slojeva neuronskih mreža do niskonivovskih tenzorskih operacija.
- `R.call_dps_packed`: funkcija u Relax-u kojom se pozivaju TIR funkcije, kojoj korisnik pored ulaznih podataka dostavlja i izlazni bafer. Funkcija upisuje rezultat u izlazni bafer koji je deo njene liste argumenata. Ova konvencija se naziva **prosleđivanje odredišta (*destination passing*)**. Ideja je da se ulazi i izlazi eksplicitno alociraju izvan funkcije i prosleđuju niskonivovskoj primitivnoj funkciji.
- `R.dataflow()`: koncepti čistoće i bočnih efekata -> čiste fje i fje sa bočnim efektima

Napomena: pogledati primere sa slajdova

LEKCIJA 5: Tenzorizacija

RestNet18 (Residual Network 18):

- Backpropagation – informacije o grešci putuju nazad kroz mrežu.
- Kod previse dubokih mreža, te informacije postaju nula dok se vrate do početnih slojeva -> nestajanje/umiranje gradijenta (i ja ću mrem dok ovo naučim :().
- RestNet18 uvodi prečice – podaci ne prolaze striktno kroz svaki sloj, već se podaci sa ulaza u neki blok slojeva kopiraju i sabiraju sa izlazom tog bloka.

Primitivne tenzorske funkcije (Primitive Tensor Functions):

To su osnovne, atomske operacije nad tenzorima (npr. pojedinačni `MatMul` ili `Conv2d`) spuštene na nivo petlji. One su osnovna jedinica optimizacije u TVM-u. Mogu imati različite apstrakcije.

TensorIR:

TensorIR je apstrakcija tenzorskih programa niskog nivoa koju koristi TVM.

Program predstavljen preko TensorIR sadrži 3 glavna elementa:

1. Višedimenzionalne bafere
2. Ugnježdene petlje
3. Blokove izračunavanja

TensorIR funkcije se kreiraju pomoću TVMScript jezika. TVMScript je jezik specifičnog domena baziran na Python-u i dizajniran je za kreiranje tenzorskih funkcija na koncizan i čitak način. U TVM-u, TensorIR funkcije su sadržane unutar IRModule i označene su dekoratorom `@T.prim_func`, i argumenti takve funkcije su višedimenzionalni baferi koji su određeni svojim identifikatorom, dimenzijama i tipom podataka koji su smešteni u njemu. Svaka `T.prim_func` funkcija mora sadržati bafere (`T.Buffer`) i/ili skalare (`T.Var`). Ukoliko ima potrebe za više ugnjeđenih petlji, može se iskoristiti `T.grid` koji omogućuje pisanje višestrukih ugnjeđenih petlji. Kontekst bloka izračunavanja se kreira pomoću `with T.block("ime_bloka")`. Glavni procesi se dešavaju u bloku izračunavanja. Blok u TensorIR programu predstavlja tenzorizovana izračunavanja na delovima višedimenzionalnih bafera. Blok izračunavanja sadrži: spoljnu petlju, potpis, init stavku i telo. Tipovi iteratora: `spatial` i `redice`. Jedna od glavnih pogodnosti TensorIR apstrakcije jeste mogućnost primene transformacija nad ugnjeđenim petljama bloka izračunavanja, bez zalaženja u telo bloka.

Primitive raspoređivanja: alat za jednostavnu primenu transformacija na petlje bloka -> `tvm.tir.Schedule`. Primitiva raspoređivanja objedinjuje skup transformacija koje se primenjuju na petlje bloka izračunavanja. Glavni cilj je podešavanje performansi. Primeri:

- `split`: Cepa jednu petlju na spoljašnju i unutrašnju na osnovu faktora.
- `tile`: Primenjuje splitovanje nad dve ose istovremeno, kreirajući 2D pločice (tiles) podataka.
- `fuse`: Spaja dve uzastopne ili ugnježdene petlje u jednu zajedničku.
- `reorder`: Menja redosled izvršavanja petlji.
- `parallel`: Paralelizacija ulaznih petlji.
- `vectorize`: Vektorizacija ulaznih petlji.

Tensor Expression DSL: za deklarativno opisivanje tenzorskih izračunavanja (npr. preko `te.compute` i `te.placeholder`) u čistom funkcionalnom jeziku. TE služi kao međukorak prilikom prevođenja Relay IR grafa izračunavanja u Tensor IR.

Napomena: pogledati promere sa slajdova

LEKCIJA 6: Portabilnost modela i ONNX format

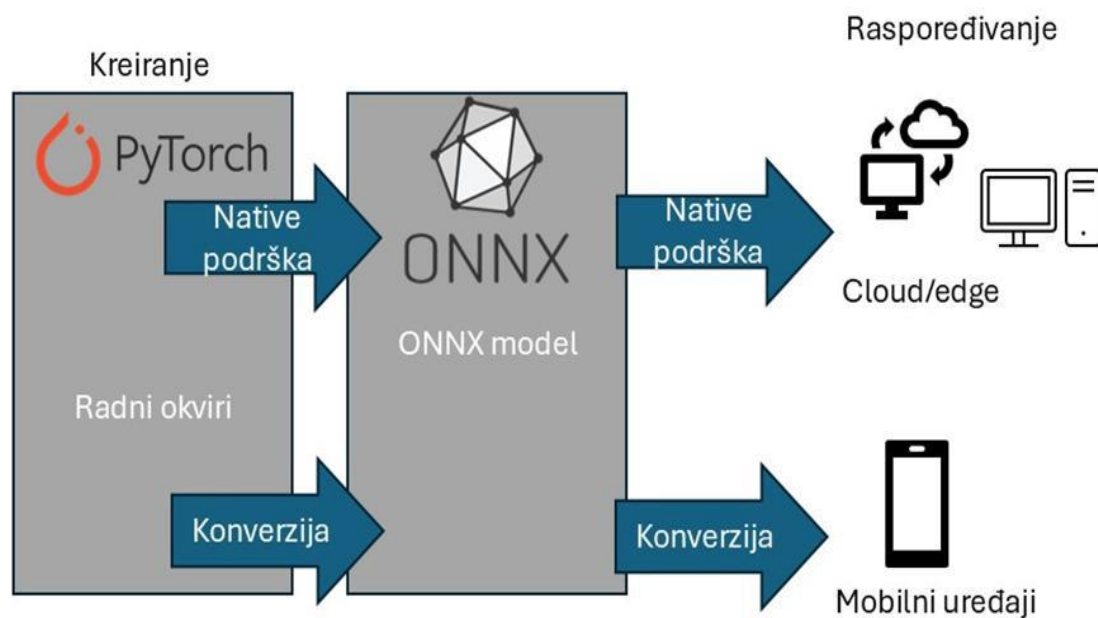
1. Jaz između okruženja razvoja i produkcije

Modeli se treniraju u Python-u na moćnim grafičkim kartama, ali se u produkciji moraju izvršavati svuda – na sporim procesorima, mobilnim telefonima, ili u C++ okruženjima bez Python zavisnosti.

2. Šta je ONNX? (Open Neural Network Exchange)

ONNX je otvoreni, standardizovani format datoteke (ekstenzija .onnx) koji definiše univerzalan, zajednički model grafa izračunavanja, omogućavajući prenos modela između bilo kog framework-a i bilo koje platforme. Ostvaranje portabilnosti pomoću ONNX ima dva glavna koraka:

- Konverzija modela iz formata specifičnog za radni okvir u ONNX format.
- Konverzija ONNX modela u format za specifičnu platformu ili radni okvir.



3. Tri stuba ONNX specifikacije

ONNX postiže univerzalnost kroz striktnu specifikaciju tri komponente:

1. Ekstenzibilni model grafova (Computational Graph Model): Standardizovana definicija usmerenih acikličnih grafova.
2. Standardni tipovi podataka: Unificirani formati za tenzore i bazične tipove podataka (Tensor proto).

3. Ugrađeni operatori (Built-in operators): Zvaničan, fiksni skup matematičkih operatora (npr. `onnx.Conv`, `onnx.Add`, `onnx.MaxPool`) koji imaju identično ponašanje na svakom hardveru.

4. Životni put modela kroz ONNX ekosistem

- Faza 1: Izvoz (Export): Prevođenje modela iz PyTorch-a preko komande `torch.onnx.export(model, dummy_input, "model.onnx")`. *Uočiti:* Prolaskom lažnog ulaza (`dummy_input`), PyTorch "crtá" i snima statičku putanju grafa. Izvoz može biti statički i dinamički.
- Faza 2: Izvršavanje (Inference): Kada imamo `.onnx` fajl, framework nam više ne treba. Model se može pokrenuti na dva načina:
 1. ONNX Runtime: Lagano C++ okruženje koje direktno i brzo interpretira ONNX graf na hardveru.
 2. Kompilacija kroz TVM: TVM koristi ONNX kao svoj frontend. TVM učitá ONNX graf, pretvori ga u svoj Relax IR modul, uradi fuziju petlji i izbaci maksimalno optimizovan sirovi mašinski kod za željeni čip.

5. ONNX format

Zajednički format za reprezentovanje modela dubokog učenja. Model se sastoji od grafa izračunavanja, pri čemu čvorovi predstavljaju operacije, a grane u grafu tok podataka. Svakom čvoru odgovara neka specifična operacija poput konvolucije, poolinga ili aktivacione funkcije. Čvorovi obuhvataju i atribut kojima se opisuje njihovo ponašanje (parametre operacija).